

Robotic OT Tracker

PowerApps Conversion Guide

A complete step-by-step guide for converting the Robotic OT Tracker
from Google Sheets to Microsoft PowerApps + SharePoint Online

Direct Patch() approach — no Power Automate required

CGH da Vinci Programme | Robotic Surgery

Document Version 2.0 | 01 Sep 2026

Contents

Chapter 1 Introduction & Prerequisites

- 1.1 Purpose of this guide
- 1.2 What you will need
- 1.3 Time estimate
- 1.4 Glossary of terms

Chapter 2 Architecture Overview

- 2.1 Current system (Google Sheets + GAS)
- 2.2 New system (SharePoint + Canvas App)
- 2.3 How the two-phase write pattern protects the audit trail

Chapter 3 SharePoint Lists Setup

- 3.1 Create the SharePoint site
- 3.2 Instruments list
- 3.3 Consumables list
- 3.4 Faults list
- 3.5 AuditTrail list
- 3.6 Index key columns for performance

Chapter 4 Build the Canvas App

- 4.1 Create the app and connect SharePoint data
- 4.2 App.OnStart — load data into collections
- 4.3 The write pattern — how every button works
- 4.4 Screen: Dashboard
- 4.5 Screen: Instruments
- 4.6 Screen: Consumables
- 4.7 Screen: Log Use
- 4.8 Screen: Manage (Change Status & Condemn)
- 4.9 Screen: Faults
- 4.10 Screen: Audit Trail
- 4.11 Navigation

Chapter 5 Data Migration from Google Sheets

- 5.1 Export from Google Sheets
- 5.2 Format for SharePoint
- 5.3 Import and verify

Chapter 6 Testing Checklist

Chapter 7 Troubleshooting

Chapter A Appendix A: SharePoint Column Quick Reference

Chapter B Appendix B: Key Power Fx Formulas

Chapter 1 — Introduction & Prerequisites

1.1 Purpose of this guide

This guide walks you through converting the **Robotic OT Tracker** into a Microsoft **PowerApps Canvas App** that stores all data in **SharePoint Online**. The Canvas App writes directly to SharePoint using built-in Power Fx formulas — **no Power Automate flows are required**.

The guide assumes you have no prior experience with PowerApps or SharePoint. Every step is numbered. Follow them from top to bottom.

1.2 What you will need

Item	What it is	Where to get it
Microsoft 365 account	A standard corporate M365 account (E1, E3, or E5).	Your IT administrator
PowerApps access	Included in M365. Go to make.powerapps.com .	make.powerapps.com
SharePoint access	Included in M365. You need permission to create a SharePoint site.	Your IT administrator
The Google Sheet	Existing tracker spreadsheet, for exporting data in Chapter 5.	Current app owner
Modern web browser	Google Chrome or Microsoft Edge (latest version).	Your computer

✓ *Power Automate is NOT needed. Everything is built inside PowerApps using built-in SharePoint connector formulas.*

1.3 Time estimate

Task	Time
Chapter 3: Create SharePoint lists	1–2 hours
Chapter 4: Build the Canvas App	6–10 hours
Chapter 5: Data migration	1–2 hours
Chapter 6: Testing	1–2 hours
Total	9–16 hours

1.4 Glossary

Term	Meaning
Canvas App	A Microsoft PowerApps app where you drag-and-drop controls and write formulas. The app nurses and staff will use.
SharePoint List	A table in SharePoint Online. Replaces Google Sheets. Has rows (items) and columns (fields).
Power Fx	The formula language used in Canvas Apps — similar to Excel formulas.
Collection	A local in-memory table inside the Canvas App. Data loads from SharePoint into a collection once at startup so the app feels fast.
Patch()	The Power Fx function that writes (creates or updates) a record in a SharePoint List.
IfError()	A Power Fx function that runs one action, then runs a different action if the first one fails. Used to safely handle write failures.
Delegation	When Canvas App sends a filter to SharePoint to run server-side. Always use delegation-safe formulas for large lists so you do not miss data.
Serial Number (SN)	Unique identifier for each instrument or consumable. Stored in the "Title" column of each SharePoint list.

Chapter 2 — Architecture Overview

2.1 Current system

Layer	Technology	Role
UI	HTML + CSS + JavaScript (GitHub Pages)	What users see in browser
Logic	Vanilla JavaScript (app.js)	Business rules, validation, state
API	Google Apps Script REST endpoint	Reads/writes Google Sheets
Database	Google Sheets (4 tabs)	Instruments, Consumables, Faults, AuditTrail

2.2 New system

Layer	Technology	Role
UI	PowerApps Canvas App	What users see and interact with — phone, tablet, or browser
Logic	Power Fx formulas in Canvas App	All business rules, validation, and write operations — directly in the app
Database	SharePoint Online (4 lists)	Instruments, Consumables, Faults, AuditTrail

✓ *There is no separate API layer. The Canvas App talks to SharePoint directly using built-in connectors. This is simpler, faster, and easier to maintain.*

2.3 The two-phase write pattern

The original app had a bug: when logging a use, it made two separate API calls. If the network dropped between them, the instrument record was updated but no audit trail entry was created — a silent failure.

In the new app, every write operation uses the **two-phase write pattern**:

Phase	What happens	If it fails
Phase 1 — Primary write	Patch() writes to the main list (e.g. Instruments). Wrapped in IfError().	Nothing was saved. Show error message. User can try again.
Phase 2 — Audit write	Only runs if Phase 1 succeeded. Patch() writes to AuditTrail. Also wrapped in IfError().	Primary was saved (instrument updated). Show a prominent WARNING with full event details for manual recording.

This means:

- If the primary write fails — **nothing** was changed. User sees an error and retries.
- If the audit write fails — the data **was** changed. User sees a specific warning with every detail they need to make a manual log entry.
- There are no silent failures. The user always knows exactly what happened.

■ *This pattern exactly mirrors the robust fix applied to the original JavaScript app — just expressed in Power Fx instead of JavaScript.*

Chapter 3 — SharePoint Lists Setup

3.1 Create the SharePoint site

1. Open a browser and go to **office.com**. Sign in with your M365 account.
2. Click the grid icon (■) top-left. Select **SharePoint**.
3. Click **+ Create site > Team site**.
4. Site name: **Robotic OT Tracker**. Privacy: **Private**. Click **Next** then **Finish**.
5. Copy the site URL from the browser — you will need it in Chapter 4. Example:
https://yourorg.sharepoint.com/sites/RoboticOTTracker

✓ Write down your site URL now. You will paste it many times in Chapter 4.

3.2 Instruments list

1. In your SharePoint site, click **New > List > Blank list**.
2. Name: **Instruments**. Click **Create**.
3. Click the column header **Title > Column settings > Rename**. Change to **SerialNumber**. Click **Save**.

■ *SharePoint stores "SerialNumber" internally as "Title". In Canvas App formulas, reference it as **ThisItem.Title** or **ThisItem.SerialNumber** — both work.*

4. Click **+ Add column** for each column below:

Column name	Type	Settings
InstrumentType	Choice	Choices: Monopolar scissors / Fenestrated bipolar / Cadiere forceps / Maryland forceps / Prograsp forceps / Curved bipolar dissector / Large needle driver / Large suturecut needle driver / Cautery hook / Cautery spatula / Fenestrated TIP-UP / 0 degree lens / 30 degree lens
Status	Choice	Choices: Circulation / SB Instrument / URO Set / Non-sterile / Complete / Condemned. Default: Circulation
UsesLeft	Number	No decimal places
MaxLife	Number	No decimal places
LastUsed	Date and time	Format: Date only
LastCase	Single line of text	Leave defaults
Remarks	Multiple lines of text	Plain text

■ Use these exact column names. The Canvas App formulas in Chapter 4 reference them directly and are case-sensitive.

3.3 Consumables list

1. Click **New > List > Blank list**. Name: **Consumables**. Click **Create**.
2. Rename "Title" to **SerialNumber**.
3. Add columns:

Column name	Type	Settings
ConsumableType	Choice	Choices: Small clip applicator / M/L clip applicator / Large clip applicator / Vessel sealer / SureForm 60 / SureForm 45 / Harmonic scalpel
Balance	Number	No decimal places
MaxBalance	Number	No decimal places
Expiry	Date and time	Format: Date only
LastUsed	Date and time	Format: Date only

3.4 Faults list

1. New > List > Blank list. Name: **Faults**. Rename Title to **SerialNumber**.
2. Add columns:

Column name	Type	Settings
FaultTimestamp	Date and time	Format: Date & Time
InstrumentType	Single line of text	Leave defaults
FaultType	Choice	Choices: Wire broken / Wire frail / Cannot grasp / Cannot close / Jaw unable to open / Intra-op failure / Call back / Cannot fire (stapler) / Display issue (lens) / Other
CaseNumber	Single line of text	Leave defaults
Notes	Multiple lines of text	Plain text
LoggedBy	Single line of text	Leave defaults

3.5 AuditTrail list

1. New > List > Blank list. Name: **AuditTrail**. Rename Title to **SerialNumber**.
2. Add columns:

Column name	Type	Settings
EventTimestamp	Date and time	Format: Date & Time
Event	Choice	Choices: Use logged / Consumable use logged / Fault logged / Use REVERSED / Status changed / Instrument condemned / Instrument added / Consumable added
InstrumentType	Single line of text	Leave defaults
Detail	Multiple lines of text	Plain text — stores the full event description
CaseNumber	Single line of text	Leave defaults
LoggedBy	Single line of text	Leave defaults

3.6 Index key columns

Indexing allows Canvas App to filter SharePoint efficiently. Without it, filters only work on the first 500 rows. Repeat for **all four lists**:

1. Open the list. Click gear icon (⚙) > **List settings**.
2. Scroll to **Columns** section. Click **Indexed columns**.
3. Click **Create a new index**. Primary column: **SerialNumber**. Click **Create**.
4. On the AuditTrail list only: create a second index with primary column **EventTimestamp**.

Chapter 4 — Build the Canvas App

4.1 Create the app and connect SharePoint

1. Go to **make.powerapps.com**. Sign in.
2. Click **+ Create > Blank app > Blank canvas app**.
3. App name: **Robotic OT Tracker**. Format: **Phone**. Click **Create**.
4. Click the **Data** icon (cylinder) in the left sidebar. Click **+ Add data**. Search **SharePoint**. Select it.
5. Paste your SharePoint site URL. Press Enter. Select all four lists: **Instruments**, **Consumables**, **Faults**, **AuditTrail**. Click **Connect**.
6. You now have four SharePoint data sources connected. They appear in the Data panel on the left.

■ If SharePoint is not listed, click "New connection" and sign in with your M365 account.

Create screens

In the left Tree View, rename "Screen1" to **scrDashboard**. Then add 6 more blank screens and rename them:

scrDashboard · scrInstruments · scrConsumables · scrLogUse · scrManage · scrFaults · scrAudit

To rename a screen: right-click it in the Tree View > **Rename**.

4.2 App.OnStart — load data

App.OnStart runs once when the app starts. Use it to load data from SharePoint into local collections so the app is fast to navigate.

1. In Tree View, click the top item: **App** (not a screen).
2. In the property dropdown (top-left, usually shows "Fill"), select **OnStart**.
3. Paste this formula into the formula bar:

```
// Load all 4 lists into local collections at startup
ClearCollect(colInstruments, Instruments);
ClearCollect(colConsumables, Consumables);
ClearCollect(colFaults, Faults);
// Audit trail: load last 90 days only (keeps it fast)
ClearCollect(colAudit,
Filter(AuditTrail,
EventTimestamp >= DateAdd(Today(), -90, TimeUnit.Days)
)
);
Set(varCurrentUser, User().DisplayName);
```

■ *ClearCollect()* downloads SharePoint data to the device. The app reads from the local collection (fast) and writes directly to SharePoint (requires network).

4.3 The write pattern — how every button works

Every write button in the app (Log Use, Change Status, Condemn, etc.) follows the **same two-phase pattern**. Understanding this template makes the rest of the chapter easy to follow.

The template

```
// --- PHASE 1: Primary write ---
Set(varWriteError, false);
IfError(
  Patch(ListName, RecordToUpdate, {Field1: value1, Field2: value2}),
  Set(varWriteError, true)
);

If(varWriteError,
  // Phase 1 failed – nothing was saved
  Notify("Save failed. Check your connection and try again.",
    NotificationType.Error),

  // Phase 1 succeeded – update local collection
  Patch(colLocalCollection, SelectedRecord, {Field1: value1, Field2: value2});

  // --- PHASE 2: Audit write ---
  Set(varAuditError, false);
  IfError(
    Patch(AuditTrail, Defaults(AuditTrail), {
      Title: serialNumber,
      EventTimestamp: Now(),
      Event: "Event name here",
      Detail: "Human-readable description of what happened",
      CaseNumber: caseNumber,
      LoggedBy: varCurrentUser
    }),
    Set(varAuditError, true)
  );

  If(varAuditError,
    // Phase 2 failed – primary was saved, audit was not
    Notify("WARNING: Use saved but AUDIT TRAIL FAILED. " &
      "Record manually: [details here]",
      NotificationType.Warning),
    // Both succeeded
    Notify("Action completed.", NotificationType.Success)
  )
)
```

The key points:

- **IfError(Patch(...), Set(varWriteError, true))** — attempts the write. If it fails, sets the error flag. Does NOT stop the formula.

- **If(varWriteError, ...)** — checks the flag and branches accordingly.
- Phase 2 only runs inside the "success" branch of Phase 1. If Phase 1 failed, Phase 2 never runs.
- The warning message for Phase 2 failure includes all details the user needs to make a manual log entry.

4.4 Screen: Dashboard

Shows summary stat cards. Five cards with counts.

1. Select **scrDashboard**. In Properties panel, set Fill = **#00606E** (teal).
2. Insert > Text label. Text = **"Robotic OT Tracker"**. Size 22, bold, white.
3. Insert > Text label. Text = **"CGH · da Vinci Programme"**. Size 12, light colour.
4. For each stat card, insert a white rectangle and two labels inside it. The number label Text formulas:

Card label	Number label Text formula
In Circulation	CountRows(Filter(colInstruments, Status = "Circulation"))
Critical (≤ 2 uses)	CountRows(Filter(colInstruments, UsesLeft <= 2 And Status <> "Condemned" And Status <> "Complete"))
SB Instruments	CountRows(Filter(colInstruments, Status = "SB Instrument"))
Faults (30 days)	CountRows(Filter(colFaults, FaultTimestamp >= DateAdd(Today(), -30, TimeUnit.Days)))
Consumables Available	CountRows(Filter(colConsumables, Balance > 0 And (IsBlank(Expiry) Or Expiry >= Today())))

5. Insert > Button. Text = **"Refresh"**. OnSelect:

```
ClearCollect(colInstruments, Instruments);
ClearCollect(colConsumables, Consumables);
ClearCollect(colFaults, Faults);
ClearCollect(colAudit,
Filter(AuditTrail,
EventTimestamp >= DateAdd(Today(), -90, TimeUnit.Days)
)
);
Notify("Data refreshed.", NotificationType.Success)
```

4.5 Screen: Instruments

1. Insert > Vertical gallery. Items:

```
Sort(
  Filter(colInstruments,
    IsBlank(txtInstSearch.Text) Or
    Title in txtInstSearch.Text Or
    InstrumentType in txtInstSearch.Text
  ),
  Title, SortOrder.Ascending
)
```

2. Inside the gallery, add labels:

Label	Text property
Serial number	ThisItem.Title
Type	ThisItem.InstrumentType
Status	ThisItem.Status
Uses remaining	Text(ThisItem.UsesLeft) & " / " & Text(ThisItem.MaxLife)
Last used	Text(ThisItem.LastUsed, "dd/mm/yyyy")

3. Status badge colour — Insert a Rectangle behind the status label. Fill:

```
Switch(ThisItem.Status,
  "Circulation", ColorValue("#2A7A2A"),
  "SB Instrument", ColorValue("#1A5A9A"),
  "URO Set", ColorValue("#7A2A9A"),
  "Non-sterile", ColorValue("#C87A00"),
  "Complete", ColorValue("#4A5568"),
  "Condemned", ColorValue("#C82020"),
  ColorValue("#4A5568")
)
```

4. Life remaining progress bar — Insert a thin Rectangle. Width:

```
If(IsBlank(ThisItem.MaxLife) Or ThisItem.MaxLife = 0, 0,
  Parent.Width * (ThisItem.UsesLeft / ThisItem.MaxLife)
)
```

5. Insert a Text input above the gallery. Name: **txtInstSearch**. HintText: "Search serial number or type..."

4.6 Screen: Consumables

1. Insert > Vertical gallery. Items:

```
Sort(
  Filter(colConsumables,
    IsBlank(txtConsSearch.Text) Or
    Title in txtConsSearch.Text Or
    ConsumableType in txtConsSearch.Text
  ),
  Title, SortOrder.Ascending
)
```

2. Inside the gallery, add labels:

Label	Text or Fill property
Serial number	ThisItem.Title
Type	ThisItem.ConsumableType
Balance	Text(ThisItem.Balance) & " / " & Text(ThisItem.MaxBalance)
Expiry	Text(ThisItem.Expiry, "dd/mm/yyyy")
Expiry label colour	If(Not IsBlank(ThisItem.Expiry) And ThisItem.Expiry < Today(), Red, If(Not IsBlank(ThisItem.Expiry) And ThisItem.Expiry < DateAdd(Today(),30,TimeUnit.Days), ColorValue("#C87A00"), RGBA(30,30,30,1)))

4.7 Screen: Log Use

This screen logs instrument and consumable use against a case number. It is the most critical screen.

Controls

Control	Name	Purpose
Text input	txtCaseNum	Case visiting number (e.g. XXX456789A)
Dropdown	drpLogType	Two options: "Instrument" and "Consumable"
Vertical gallery	galLogItems	List of instruments or consumables to select from
Button	btnLogUse	"Log Use" — executes the write

Gallery Items formula

```
If(drpLogType.Selected.Value = "Instrument",
  Filter(colInstruments,
    Status <> "Condemned" And Status <> "Complete"),
  Filter(colConsumables, Balance > 0)
)
```

Log Use button — OnSelect (instrument path)

First, validate and check the same-case lock:

```
// Validate
If(IsBlank(galLogItems.Selected) Or IsBlank(txtCaseNum.Text),
  Notify("Select an item and enter a case number.", NotificationType.Warning),
  // Same-case lock (instruments only)
  If(drpLogType.Selected.Value = "Instrument" And
    galLogItems.Selected.LastCase = txtCaseNum.Text,
    Notify("Already logged for case " & txtCaseNum.Text,
      NotificationType.Warning),
    // Proceed with write (see below)
    Set(varNewUses, galLogItems.Selected.UsesLeft - 1);
    Set(varNewStatus,
      If(varNewUses = 0, "Complete", galLogItems.Selected.Status)
    );
```

Then the two-phase write (paste this continuing inside the validation If above):


```
// ■■ PHASE 1: Update Instruments ■■
Set(varWriteError, false);
IfError(
  Patch(Instruments, galLogItems.Selected, {
    UsesLeft: varNewUses,
    LastUsed: Today(),
    LastCase: txtCaseNum.Text,
    Status: varNewStatus
  }),
  Set(varWriteError, true)
);
If(varWriteError,
  Notify("Save failed. Check connection.", NotificationType.Error),
  // ■■ Primary saved – update local collection ■■
  Patch(colInstruments, galLogItems.Selected, {
    UsesLeft: varNewUses,
    LastUsed: Today(),
    LastCase: txtCaseNum.Text,
    Status: varNewStatus
  });
  // ■■ PHASE 2: Audit trail ■■
  Set(varAuditError, false);
  IfError(
    Patch(AuditTrail, Defaults(AuditTrail), {
      Title: galLogItems.Selected.Title,
      EventTimestamp: Now(),
      Event: "Use logged",
      InstrumentType: galLogItems.Selected.InstrumentType,
      Detail: "Uses left: " & varNewUses &
        ", Case: " & txtCaseNum.Text &
        If(varNewStatus="Complete", " – REACHED MAX LIFE", ""),
      CaseNumber: txtCaseNum.Text,
      LoggedBy: varCurrentUser
    }),
    Set(varAuditError, true)
  );
  If(varAuditError,
    Notify("WARNING: Use saved – AUDIT TRAIL FAILED. " &
      "Record manually: SN " & galLogItems.Selected.Title &
      " used in case " & txtCaseNum.Text &
      " at " & Text(Now(),"dd/mm/yyyy hh:mm"),
      NotificationType.Warning),
    Notify("Use logged successfully.", NotificationType.Success)
  )
)
) // closes same-case lock
) // closes input validation
```

Log Use button — consumable path

For consumables, the pattern is the same but simpler (no same-case lock, no status change). Add this as an If() branch for the consumable case:

```
// Consumable path – inside drpLogType check
Set(varNewBalance, galLogItems.Selected.Balance - 1);
Set(varWriteError, false);
IfError(
  Patch(Consumables, galLogItems.Selected, {
    Balance: varNewBalance,
    LastUsed: Today()
  }),
  Set(varWriteError, true)
);
If(varWriteError,
  Notify("Save failed. Check connection.", NotificationType.Error),
  Patch(colConsumables, galLogItems.Selected, {
    Balance: varNewBalance, LastUsed: Today()
  });
Set(varAuditError, false);
IfError(
  Patch(AuditTrail, Defaults(AuditTrail), {
    Title: galLogItems.Selected.Title,
    EventTimestamp: Now(),
    Event: "Consumable use logged",
    InstrumentType: galLogItems.Selected.ConsumableType,
    Detail: "Balance after: " & varNewBalance &
    ", Case: " & txtCaseNum.Text,
    CaseNumber: txtCaseNum.Text,
    LoggedBy: varCurrentUser
  }),
  Set(varAuditError, true)
);
If(varAuditError,
  Notify("WARNING: Consumable saved – AUDIT FAILED. Record manually: " &
  galLogItems.Selected.Title & " used in case " & txtCaseNum.Text,
  NotificationType.Warning),
  Notify("Consumable use logged.", NotificationType.Success)
)
)
```

4.8 Screen: Manage

Two panels: Change Status and Condemn Instrument.

Change Status

Add a gallery of active instruments, a dropdown for new status, and a button:

```
// Change Status button OnSelect
Set(varWriteError, false);
IfError(
  Patch(Instruments, galManageInst.Selected,
    {Status: drpNewStatus.Selected.Value}),
  Set(varWriteError, true)
);
If(varWriteError,
  Notify("Save failed.", NotificationType.Error),
  Patch(colInstruments, galManageInst.Selected,
    {Status: drpNewStatus.Selected.Value});
  Set(varAuditError, false);
  IfError(
    Patch(AuditTrail, Defaults(AuditTrail), {
      Title: galManageInst.Selected.Title,
      EventTimestamp: Now(),
      Event: "Status changed",
      Detail: "New status: " & drpNewStatus.Selected.Value,
      LoggedBy: varCurrentUser
    }),
    Set(varAuditError, true)
  );
  If(varAuditError,
    Notify("Status updated – audit trail failed. Record manually: SN " &
      galManageInst.Selected.Title & " status changed to " &
      drpNewStatus.Selected.Value, NotificationType.Warning),
    Notify("Status updated.", NotificationType.Success)
  )
)
```

Condemn Instrument

Add a gallery and a dropdown for condemn reason (choices: End of life — max uses reached / Intra-op failure / Physical damage / Call back by vendor / Other). Button:

```
// Condemn button OnSelect
Set(varWriteError, false);
IfError(
    Patch(Instruments, galCondemnInst.Selected, {Status: "Condemned"}),
    Set(varWriteError, true)
);
If(varWriteError,
    Notify("Save failed.", NotificationType.Error),
    Patch(colInstruments, galCondemnInst.Selected, {Status: "Condemned"});
    Set(varAuditError, false);
    IfError(
        Patch(AuditTrail, Defaults(AuditTrail), {
            Title: galCondemnInst.Selected.Title,
            EventTimestamp: Now(),
            Event: "Instrument condemned",
            Detail: "Reason: " & drpCondemnReason.Selected.Value,
            LoggedBy: varCurrentUser
        }),
        Set(varAuditError, true)
    );
    If(varAuditError,
        Notify("Condemned – audit trail failed. Record manually: SN " &
            galCondemnInst.Selected.Title & " condemned, reason: " &
            drpCondemnReason.Selected.Value, NotificationType.Warning),
        Notify("Instrument condemned.", NotificationType.Success)
    )
)
```

4.9 Screen: Faults

A form to log a fault, and a gallery showing recent faults.

Control	Name	Purpose
Gallery	galFaultInst	Select the instrument. Items: Filter(colInstruments, Status <> "Condemned")
Dropdown	drpFaultType	Fault type (Wire broken / Wire frail / Cannot grasp / Cannot close / Jaw unable to open / Intra-op failure / Call back / Cannot fire (stapler) / Display issue (lens) / Other)
Text input	txtFaultCase	Case number
Text input (multiline)	txtFaultNotes	Additional notes
Button	btnLogFault	"Log Fault"

Log Fault button OnSelect:

```

If(IsBlank(galFaultInst.Selected) Or IsBlank(drpFaultType.Selected.Value),
Notify("Select instrument and fault type.", NotificationType.Warning),
// Phase 1: create fault record
Set(varWriteError, false);
IfError(
Patch(Faults, Defaults(Faults), {
Title: galFaultInst.Selected.Title,
FaultTimestamp: Now(),
InstrumentType: galFaultInst.Selected.InstrumentType,
FaultType: drpFaultType.Selected.Value,
CaseNumber: txtFaultCase.Text,
Notes: txtFaultNotes.Text,
LoggedBy: varCurrentUser
}),
Set(varWriteError, true)
);
If(varWriteError,
Notify("Save failed.", NotificationType.Error),
// Update local collection
Collect(colFaults, {
Title: galFaultInst.Selected.Title,
FaultType: drpFaultType.Selected.Value,
FaultTimestamp: Now(),
CaseNumber: txtFaultCase.Text
});
// Phase 2: audit
Set(varAuditError, false);
IfError(
Patch(AuditTrail, Defaults(AuditTrail), {
Title: galFaultInst.Selected.Title,
EventTimestamp: Now(),
Event: "Fault logged",
Detail: drpFaultType.Selected.Value & ", Case: " & txtFaultCase.Text,
CaseNumber: txtFaultCase.Text,
LoggedBy: varCurrentUser
}),
Set(varAuditError, true)
);
If(varAuditError,
Notify("Fault saved – audit failed. Record manually: " &
galFaultInst.Selected.Title & " fault logged.",
NotificationType.Warning),
Notify("Fault logged.", NotificationType.Success)
)
)
)

```

4.10 Screen: Audit Trail

1. Insert a **Date picker** named **dtAuditFrom**. Default value: **DateAdd(Today(), -7, TimeUnit.Days)**
2. Insert a **Date picker** named **dtAuditTo**. Default value: **Today()**
3. Insert a **Dropdown** named **drpAuditEvent**. Items:

```
[ "", "Use logged", "Consumable use logged", "Fault logged", "Use REVERSED",
  "Status changed", "Instrument condemned", "Instrument added", "Consumable added" ]
```

4. Insert a **Text input** named **txtAuditSearch** for serial number search.

5. Insert a Vertical gallery. Items:

```
SortByColumns(
  Filter(colAudit,
    (IsBlank(drpAuditEvent.Selected.Value) Or
    Event = drpAuditEvent.Selected.Value)
    And
    (IsBlank(txtAuditSearch.Text) Or Title in txtAuditSearch.Text)
    And EventTimestamp >= dtAuditFrom.SelectedDate
    And EventTimestamp <= DateAdd(dtAuditTo.SelectedDate, 1, TimeUnit.Days)
  ),
  "EventTimestamp", Descending
)
```

6. Set the screen **OnVisible** to reload the audit data when the screen opens:

```
ClearCollect(colAudit,
  Filter(AuditTrail,
    EventTimestamp >= dtAuditFrom.SelectedDate
  )
);
```

7. In the gallery, show labels for: **Title (SN)**, **Event**, **Detail**, **CaseNumber**, **LoggedBy**, **Text(EventTimestamp, "dd/mm/yyyy hh:mm")**.

4.11 Navigation

Add navigation buttons at the bottom of every screen. Design one set, copy and paste it to each screen.

Button	OnSelect
Dashboard	Navigate(scrDashboard, ScreenTransition.None)
Instruments	Navigate(scrInstruments, ScreenTransition.None)
Consumables	Navigate(scrConsumables, ScreenTransition.None)
Log Use	Navigate(scrLogUse, ScreenTransition.None)
Manage	Navigate(scrManage, ScreenTransition.None)
Faults	Navigate(scrFaults, ScreenTransition.None)
Audit	Navigate(scrAudit, ScreenTransition.None)

✓ After connecting navigation, publish the app with **File > Save > Publish**. Test on your phone via the PowerApps mobile app (App Store / Google Play).

Chapter 5 — Data Migration from Google Sheets

5.1 Export from Google Sheets

1. Open the existing Robotic OT Tracker Google Sheet.
2. Click the **Instruments** tab > **File** > **Download** > **Microsoft Excel (.xlsx)**. Save as **instruments_export.xlsx**.
3. Repeat for the **Consumables** tab: save as **consumables_export.xlsx**.

■ You only need to migrate Instruments and Consumables. Decide with your team whether to migrate Faults and AuditTrail, or start fresh.

5.2 Format for SharePoint

Instruments — rename header row columns to:

Google Sheets column	Rename to
SN / Serial	Title
Type	InstrumentType
Status	Status
Uses Left	UsesLeft
Max Life	MaxLife
Last Used	LastUsed
Last Case	LastCase
Remarks	Remarks

Consumables — rename header row columns to:

Google Sheets column	Rename to
SN / Serial	Title
Type	ConsumableType
Balance	Balance
Max Balance	MaxBalance
Expiry	Expiry
Last Used	LastUsed

■ **Status values must match your Choice options exactly: "Circulation", "SB Instrument", "URO Set", "Non-sterile", "Complete", "Condemned". Fix any differences before importing.**

5.3 Import and verify

1. Open your Instruments list in SharePoint.
2. Click the grid/edit icon to enter **Quick Edit** mode.
3. In Excel, select all data rows (not the header). Copy with Ctrl+C.
4. In SharePoint Quick Edit, click the first empty cell in the Title column. Paste with Ctrl+V.
5. Review for errors (red cells = problem, usually a Choice value mismatch). Click **Exit grid view** to save.
6. Repeat for the Consumables list.
7. Open the Canvas App. Check the Dashboard counts and browse the Instruments screen to confirm all data is present and correct.

■ **Run the new app in parallel with the Google Sheets app for at least two weeks before switching over fully.**

Chapter 6 — Testing Checklist

Complete all items before going live. Test on a real device (phone or tablet), not just the browser preview.

#	Test	Expected result	Done
1	Dashboard loads	All 5 stat cards show correct counts	■
2	Log Use: instrument, valid case number	Use logged. UsesLeft -1. Audit entry created.	■
3	Log Use: same case number twice for same instrument	Warning: already logged for this case.	■
4	Log Use: blank case number	Warning: enter a case number.	■
5	Log Consumable: consumable, case number	Balance -1. Audit entry created.	■
6	Log Use: instrument with 1 use remaining	UsesLeft becomes 0. Status changes to Complete. No longer in Log Use list.	■
7	Instruments screen: search by serial number	Gallery filters to matching row.	■
8	Manage: change status of instrument	Status badge updates. Audit entry created.	■
9	Manage: condemn instrument	Status = Condemned. No longer in Log Use list. Audit entry created.	■
10	Faults: log a fault	Fault appears in gallery. Audit entry created.	■
11	Audit Trail: filter by event type	Only matching events shown.	■
12	Audit Trail: change date range	Gallery updates correctly.	■
13	Turn off Wi-Fi, try Log Use	Error: save failed. Check connection.	■
14	Restore Wi-Fi, try Log Use again	Succeeds normally. No duplicate entries.	■
15	Simulate audit fail (disconnect after primary, before audit write)	WARNING message with all manual-log details. Primary change was saved.	■
16	Two devices: log use on one, refresh on other	Second device shows updated data after refresh.	■

Chapter 7 — Troubleshooting

Symptom	Likely cause	Fix
Patch() always shows error; data never saves	SharePoint connection expired or disconnected	In PowerApps Studio, open the Data panel. Remove and re-add the SharePoint connection. Re-enter the site URL.
Formula bar shows red underline on column name (e.g. "UsesLeft")	Column name typo or case mismatch	Check the exact column name in SharePoint (List Settings > Columns). Names are case-sensitive. "UsesLeft" and "usesleft" are different.
Gallery shows "Delegation warning" triangle	Filter uses a non-delegable function or a non-indexed column	Ensure SerialNumber (Title) is indexed (Chapter 3.6). Use only =, <>, <, >, >=, <=, StartsWith with indexed columns in Filter(). Avoid Text(), Mid(), Concatenate() inside Filter().
Dashboard counts are wrong after logging a use	Local collection (colInstruments) was not Patch()-ed after the SharePoint write	Make sure your OnSelect formula patches both Instruments (SharePoint) AND colInstruments (local collection). The gallery shows the local collection, not SharePoint directly.
Audit entries appear in AuditTrail list but not in the app Audit screen	colAudit collection was last loaded before the new entries were written	The Audit screen OnVisible formula reloads colAudit. Navigate away and back to the audit screen to trigger the reload.
Choice field (Status, Event, FaultType) rejects pasted values in Quick Edit	Pasted text does not exactly match a defined Choice option	Open the list column settings and compare your text to the exact choice options. A trailing space or wrong capitalisation will cause rejection.
App is slow on startup	ClearCollect() is loading a very large AuditTrail list	Reduce the date range in the AuditTrail ClearCollect filter (App.OnStart). 90 days is the recommended default; reduce to 30 days if the list is very large.
IfError() does not catch the Patch() failure	Power Fx IfError() works differently from try/catch — it catches error values, but some Patch() failures surface differently	Wrap with Set(varError,false); IfError(Patch(...), Set(varError,true)); If(varError,...) — this pattern reliably catches all Patch() failures.

Appendix A — SharePoint Column Quick Reference

Instruments

Display name	Internal name (in formulas)	Type	Index?
SerialNumber	Title (built-in, renamed)	Single line of text	Yes
InstrumentType	InstrumentType	Choice	No
Status	Status	Choice	No
UsesLeft	UsesLeft	Number	No
MaxLife	MaxLife	Number	No
LastUsed	LastUsed	Date and Time (date only)	No
LastCase	LastCase	Single line of text	No
Remarks	Remarks	Multiple lines of text	No

Consumables

Display name	Internal name (in formulas)	Type	Index?
SerialNumber	Title (built-in, renamed)	Single line of text	Yes
ConsumableType	ConsumableType	Choice	No
Balance	Balance	Number	No
MaxBalance	MaxBalance	Number	No
Expiry	Expiry	Date and Time (date only)	No
LastUsed	LastUsed	Date and Time (date only)	No

Faults

Display name	Internal name (in formulas)	Type	Index?
SerialNumber	Title (built-in, renamed)	Single line of text	Yes
FaultTimestamp	FaultTimestamp	Date and Time	No
InstrumentType	InstrumentType	Single line of text	No
FaultType	FaultType	Choice	No
CaseNumber	CaseNumber	Single line of text	No

Display name	Internal name (in formulas)	Type	Index?
Notes	Notes	Multiple lines of text	No
LoggedBy	LoggedBy	Single line of text	No

AuditTrail

Display name	Internal name (in formulas)	Type	Index?
SerialNumber	Title (built-in, renamed)	Single line of text	Yes
EventTimestamp	EventTimestamp	Date and Time	No
Event	Event	Choice	No
InstrumentType	InstrumentType	Single line of text	No
Detail	Detail	Multiple lines of text	No
CaseNumber	CaseNumber	Single line of text	No
LoggedBy	LoggedBy	Single line of text	No

Appendix B — Key Power Fx Formulas

App.OnStart

```
ClearCollect(colInstruments, Instruments);
ClearCollect(colConsumables, Consumables);
ClearCollect(colFaults, Faults);
ClearCollect(colAudit,
Filter(AuditTrail,
EventTimestamp >= DateAdd(Today(), -90, TimeUnit.Days)));
Set(varCurrentUser, User().DisplayName);
```

Two-phase write pattern (the template for every button)

```
Set(varWriteError, false);
IfError(Patch(ListName, Record, {field: value}), Set(varWriteError, true));
If(varWriteError,
Notify("Save failed.", NotificationType.Error),
Patch(colLocalCollection, Record, {field: value}); // update local
Set(varAuditError, false);
IfError(
Patch(AuditTrail, Defaults(AuditTrail), {Title: sn, Event: "...", ...}),
Set(varAuditError, true)
);
If(varAuditError,
Notify("WARNING: saved but audit failed. Record manually.", NotificationType.Warning),
Notify("Done.", NotificationType.Success)
)
)
```

Dashboard stat cards

Card	Formula
In Circulation	CountRows(Filter(colInstruments, Status = "Circulation"))
Critical	CountRows(Filter(colInstruments, UsesLeft <= 2 And Status <> "Condemned" And Status <> "Complete"))
SB Instruments	CountRows(Filter(colInstruments, Status = "SB Instrument"))
Faults 30 days	CountRows(Filter(colFaults, FaultTimestamp >= DateAdd(Today(), -30, TimeUnit.Days)))
Consumables available	CountRows(Filter(colConsumables, Balance > 0 And (IsBlank(Expiry) Or Expiry >= Today())))

Status badge colour

```
Switch(ThisItem.Status,
"Circulation", ColorValue("#2A7A2A"),
"SB Instrument", ColorValue("#1A5A9A"),
"URO Set", ColorValue("#7A2A9A"),
"Non-sterile", ColorValue("#C87A00"),
"Complete", ColorValue("#4A5568"),
"Condemned", ColorValue("#C82020"),
ColorValue("#4A5568"))
```

Life remaining progress bar

```
If(IsBlank(ThisItem.MaxLife) Or ThisItem.MaxLife = 0, 0,
Parent.Width * (ThisItem.UsesLeft / ThisItem.MaxLife))
```

Expiry warning colour

```
If(Not IsBlank(ThisItem.Expiry) And ThisItem.Expiry < Today(), Red,
If(Not IsBlank(ThisItem.Expiry) And
ThisItem.Expiry < DateAdd(Today(), 30, TimeUnit.Days),
ColorValue("#C87A00"),
RGBA(30, 30, 30, 1)))
```

Log Use gallery items

```
If(drpLogType.Selected.Value = "Instrument",
Filter(colInstruments, Status <> "Condemned" And Status <> "Complete"),
Filter(colConsumables, Balance > 0))
```

Audit Trail gallery items

```
SortByColumns(
Filter(colAudit,
(IsBlank(drpAuditEvent.Selected.Value) Or Event = drpAuditEvent.Selected.Value)
And (IsBlank(txtAuditSearch.Text) Or Title in txtAuditSearch.Text)
And EventTimestamp >= dtAuditFrom.SelectedDate
And EventTimestamp <= DateAdd(dtAuditTo.SelectedDate, 1, TimeUnit.Days)
),
"EventTimestamp", Descending)
```

Navigation

```
Navigate(scrDashboard, ScreenTransition.None)
```